

This is a copy of a shared ChatGPT conversation
[Report conversation](#)

i will give you leetcode questions you have to do

1. explain the questions in simple points
2. explain the brute force approach in points with time complexity and space
3. give the code in c++ of that approach
4. explain better approach in points with time complexity and space
5. give the code in c++ of that approach
6. explain optimal approach in points time complexity and space
7. give the code in c++ of that approach

Done 👍

For every LeetCode question you send, I'll follow exactly this format:

1. Question Explanation

- Explain the problem in simple language.
- Explain input/output.
- Explain what is actually being asked.

2. Brute Force Approach

Idea

- Point-wise explanation.

Time Complexity

- $O(?)$

Space Complexity

- $O(?)$

C++ Code

 C++



3. Better Approach

Idea

- Point-wise explanation.

Time Complexity

- $O(?)$

Space Complexity

- $O(?)$

C++ Code

 C++



4. Optimal Approach

Idea

- Point-wise explanation.
- Why it is optimal.
- Dry run if needed.

Time Complexity

- $O(?)$

Space Complexity

- $O(?)$

C++ Code

 C++



5. Interview Notes (Optional)

- Important observation.
- Common mistakes.
- Follow-up questions interviewer may ask.

Send the first LeetCode problem. 🚀

Valid Parentheses

Min Stack

Evaluate Reverse Polish Notation

Daily Temperatures

Car Fleet

Largest Rectangle In Histogram - cover each one by one

Let's do them one by one. Starting with:

20. Valid Parentheses

1. Question Explanation

You are given a string containing only:

- ()
- { }
- []

Return `true` if:

- Every opening bracket has a corresponding closing bracket.
- Brackets close in the correct order.

Examples

Input: "()"

Output: true



Input: "()[]{}"

Output: true



Input: "(]"

Output: false



Input: "([)]"

Output: false



Because `[` should be closed before `(`.

2. Brute Force Approach

Idea

Repeatedly remove valid pairs:

- Remove `"()"`
- Remove `"{}"`
- Remove `"[]"`

Keep doing this until:

- No more removals possible.

If string becomes empty → valid.

Otherwise → invalid.

Time Complexity

Each replacement scans string.

Worst case:

$O(N^2)$



Space Complexity

$O(N)$



(new strings formed repeatedly)

C++ Code

```
class Solution {
public:
    bool isValid(string s) {

        while (true) {

            int len = s.length();

            size_t pos;

            while ((pos = s.find("(")) != string::npos)
                s.erase(pos, 2);

            while ((pos = s.find("{}")) != string::npos)
                s.erase(pos, 2);

            while ((pos = s.find("[]")) != string::npos)
                s.erase(pos, 2);

            if (s.length() == len)
                break;

        }

        return s.empty();
    }
};
```

3. Better Approach

Idea

Use a stack.

Observation

Whenever we see an opening bracket:

```
(
{
[
```



store it.

Whenever we see a closing bracket:

```
)  
}  
]
```



check whether stack top contains its matching opening bracket.

If not → invalid.

Example

```
s = "([])"
```



```
(  -> push  
[  -> push  
]  -> matches [, pop  
)  -> matches (, pop
```



Stack becomes empty.

Answer = true.

Time Complexity

```
O(N)
```



Each character processed once.

Space Complexity

```
O(N)
```



Stack may store all characters.

C++ Code

```
class Solution {
public:
    bool isValid(string s) {

        stack<char> st;

        for(char ch : s) {

            if(ch == '(' || ch == '{' || ch == '[') {
                st.push(ch);
            }
            else {

                if(st.empty())
                    return false;

                if(ch == ')' && st.top() != '(')
                    return false;

                if(ch == '}' && st.top() != '{')
                    return false;

                if(ch == ']' && st.top() != '[')
                    return false;

                st.pop();
            }
        }

        return st.empty();
    }
};
```

4. Optimal Approach

For interviews, the stack solution is already considered **optimal**.

No solution can do better than:

$O(N)$



because every character must be checked.

Idea

Instead of storing opening brackets and checking multiple conditions later:

Store the **expected closing bracket**.

Example

When we see:

(



push:

)



When we see:

[



push:

]



When we see:

{



push:

}



Now when a closing bracket arrives:

- Stack top should exactly match it.
- Otherwise invalid.

Code becomes cleaner.

Time Complexity

$O(N)$



Space Complexity

$O(N)$



C++ Code

```
class Solution {
public:
    bool isValid(string s) {

        stack<char> st;

        for(char ch : s) {

            if(ch == '(')
                st.push(')');

            else if(ch == '{')
                st.push('}');

            else if(ch == '[')
                st.push(']');

            else {

                if(st.empty() || st.top() != ch)
                    return false;

                st.pop();
            }
        }
    }
}
```

```
        return st.empty();  
    }  
};
```

Interview Notes

Common Mistakes

✗ Forgetting:

```
</> C++
```



before accessing `top()`.

✗ Returning true immediately after matching pairs.

Need to ensure:

```
</> C++
```



at the end.

Pattern Recognition

Whenever question says:

- nearest greater
- matching symbols
- undo operation
- nested structures
- balanced expression

Think:



Next: Min Stack.

yes

155. Min Stack

1. Question Explanation

Design a stack that supports:

Operations

```
push(x)  
pop()  
top()  
getMin()
```



Requirement

All operations should work efficiently.

Example

```
push(-2)  
push(0)  
push(-3)  
  
getMin() -> -3  
  
pop()  
  
top() -> 0  
  
getMin() -> -2
```



2. Brute Force Approach

Idea

Use a normal stack.

Whenever `getMin()` is called:

- Traverse entire stack.
- Find minimum element.

Example

Stack:

```
5
2
8
1
```



For `getMin()`:

```
min = 5
min = 2
min = 1
```



Return `1`.

Time Complexity

Operation	Complexity
push	$O(1)$
pop	$O(1)$
top	$O(1)$
getMin	$O(N)$

Space Complexity

$O(N)$



C++ Code

```
class MinStack {
public:

    stack<int> st;

    MinStack() {}

    void push(int val) {
        st.push(val);
    }

    void pop() {
        st.pop();
    }

    int top() {
        return st.top();
    }

    int getMin() {

        stack<int> temp = st;

        int mn = INT_MAX;

        while(!temp.empty()) {
            mn = min(mn, temp.top());
            temp.pop();
        }

        return mn;
    }
};
```

3. Better Approach

Idea

Maintain two stacks.

Stack 1

Stores all elements.

```
mainStack
```



Stack 2

Stores minimum till that point.

```
minStack
```



Example

push(5)

```
main : 5  
min  : 5
```



push(2)

```
main : 5 2  
min  : 5 2
```



push(8)

```
main : 5 2 8  
min  : 5 2
```



8 is not a new minimum.

push(1)

```
main : 5 2 8 1
min  : 5 2 1
```



pop()

If popped element equals minStack top:

Pop from both stacks.

Time Complexity

Operation	Complexity
push	$O(1)$
pop	$O(1)$
top	$O(1)$
getMin	$O(1)$

Space Complexity

$O(N)$



Extra stack.

C++ Code

```

class MinStack {
public:

    stack<int> st;
    stack<int> mn;

    MinStack() {}

    void push(int val) {

        st.push(val);

        if(mn.empty() || val <= mn.top())
            mn.push(val);
    }

    void pop() {

        if(st.top() == mn.top())
            mn.pop();

        st.pop();
    }

    int top() {
        return st.top();
    }

    int getMin() {
        return mn.top();
    }
};

```

4. Optimal Approach

This is the famous interview solution.

Use one stack only.

Store:

actual value



or

encoded value



whenever a new minimum arrives.

Key Idea

Maintain:

```
</> C++
```



Push Normal Element

If:

```
val >= mini
```



Push normally.

Push New Minimum

Suppose:

```
mini = 5  
val = 2
```



Push:

```
2*val - mini  
  
= 2*2 - 5  
  
= -1
```



Store:

-1



Update:

mini = 2



How to Detect?

Whenever:

top < mini



it means encoded value exists.

Recover Previous Minimum

Formula:

oldMin = 2*mini - encoded



Example

Push:

5
2



Stack:

5
-1



mini = 2

Pop:

```
encoded = -1
oldMin = 2*2 - (-1)
        = 5
```



Restore previous minimum.

Time Complexity

Operation	Complexity
push	O(1)
pop	O(1)
top	O(1)
getMin	O(1)

Space Complexity

O(1)



(No extra stack)

C++ Code

```
class MinStack {
public:

    stack<long long> st;
    long long mini;

    MinStack() {}

    void push(int val) {
```

```

        if(st.empty()) {
            mini = val;
            st.push(val);
        }
        else {

            if(val >= mini) {
                st.push(val);
            }
            else {

                st.push(2LL * val - mini);
                mini = val;
            }
        }
    }

void pop() {

    if(st.empty()) return;

    long long curr = st.top();
    st.pop();

    if(curr < mini) {
        mini = 2LL * mini - curr;
    }
}

int top() {

    long long curr = st.top();

    if(curr < mini)
        return mini;

    return curr;
}

int getMin() {
    return mini;
}

};

```

Most Expected Solution

For LeetCode:

Two Stack Solution



is usually accepted as the standard answer.

Follow-up Asked By Interviewers

Can you do it using:

Only one stack?



Then use the encoded-value trick.

Pattern Recognition

If a question asks:

Stack +
Current Minimum



Think:

Auxiliary Min Stack



or

Encoding Technique



Next: Evaluate Reverse Polish Notation (LC 150) — another classic stack question.

Evaluate Reverse Polish Notation (LC 150)

150. Evaluate Reverse Polish Notation

1. Question Explanation

You are given an array of strings representing an expression in **Reverse Polish Notation** (**Postfix Expression**).

Valid operators:

+
-
*
/



Return the value of the expression.

What is Reverse Polish Notation?

Normally:

2 + 3



In postfix:

2 3 +



Operator comes **after** operands.

Example 1

Input:

["2", "1", "+", "3", "*"]



Expression:

(2 + 1) * 3

Output:

9

Example 2

Input:

```
["4", "13", "5", "/", "+"]
```



Expression:

```
4 + (13 / 5)
```

Output:

```
6
```

Because:

```
13 / 5 = 2
```

```
4 + 2 = 6
```



(Integer division)

2. Brute Force Approach

Idea

Repeatedly find an operator.

When operator found:

1. Take previous two numbers.
 2. Evaluate.
 3. Replace those 3 tokens with result.
 4. Continue until one token remains.
-

Example

```
2 1 + 3 *
```



Find '+'

```
3 3 *
```



Find '*'

9



Answer = 9

Time Complexity

Every replacement shifts array.

$O(N^2)$



Space Complexity

$O(N)$



C++ Code

```
class Solution {
public:
    int evalRPN(vector<string>& tokens) {

        vector<string> arr = tokens;

        while(arr.size() > 1) {

            for(int i = 0; i < arr.size(); i++) {

                if(arr[i] == "+" || arr[i] == "-" ||
                   arr[i] == "*" || arr[i] == "/") {

                    int a = stoi(arr[i-2]);
                    int b = stoi(arr[i-1]);

                    int res;

                    if(arr[i] == "+") res = a + b;
```



```

        else if(arr[i] == "-") res = a - b;
        else if(arr[i] == "*") res = a * b;
        else res = a / b;

        arr.erase(arr.begin() + i - 2,
                  arr.begin() + i + 1);

        arr.insert(arr.begin() + i - 2,
                  to_string(res));

        break;
    }
}

return stoi(arr[0]);
}
};

```

3. Better Approach

Observation

Whenever we encounter an operator:

```

+
-
*
/

```



We need the **last two numbers**.

This is exactly what a stack provides.

Idea

Traverse tokens:

If token is a number

Push into stack.

If token is an operator

Pop:

```
b = top  
a = second top
```



Compute:

```
a op b
```



Push result back.

Dry Run

```
["2", "1", "+", "3", "*"]
```



2

Push

```
2
```



1

Push

```
2 1
```



+

Pop:

```
2 , 1
```



Compute:

2+1=3



Push

3



3

Push

3 3



*

Pop

3*3=9



Push

9



Answer = 9

Time Complexity

$O(N)$



Space Complexity

$O(N)$



C++ Code

```
class Solution {
public:
    int evalRPN(vector<string>& tokens) {

        stack<int> st;

        for(string token : tokens) {

            if(token == "+" || token == "-" ||
               token == "*" || token == "/") {

                int b = st.top();
                st.pop();

                int a = st.top();
                st.pop();

                if(token == "+")
                    st.push(a + b);

                else if(token == "-")
                    st.push(a - b);

                else if(token == "*")
                    st.push(a * b);

                else
                    st.push(a / b);
            }
            else {
                st.push(stoi(token));
            }
        }

        return st.top();
    }
};
```

4. Optimal Approach

The stack solution is already the optimal solution.

Why?

- Every token must be visited at least once.
- We process each token exactly once.

Thus:

Time = $O(N)$



is the best possible.

Cleaner Optimal Code

Use helper function.

Time Complexity

$O(N)$



Space Complexity

$O(N)$



C++ Code

```
class Solution {
public:

    int eval(int a, int b, string op) {

        if(op == "+") return a + b;
        if(op == "-") return a - b;
        if(op == "*") return a * b;
```

```

        return a / b;
    }

    int evalRPN(vector<string>& tokens) {

        stack<int> st;

        for(auto &token : tokens) {

            if(token == "+" || token == "-" ||
               token == "*" || token == "/" ) {

                int b = st.top(); st.pop();
                int a = st.top(); st.pop();

                st.push(eval(a, b, token));
            }
            else {
                st.push(stoi(token));
            }
        }

        return st.top();
    }
};

```

Interview Notes

Most Common Mistake

Many people do:

```

int a = st.top(); st.pop();
int b = st.top(); st.pop();

```

```

// ...

```

```

// C++

```

Wrong.

For:



Correct evaluation:

4 - 2 = 2



Therefore:

```
int b = st.top(); st.pop();  
int a = st.top(); st.pop();
```

```
return a - b;
```

```
// C++
```



Always remember:

Second Top OP Top



Pattern Recognition

If a question involves:

Postfix Expression
Prefix Expression
Expression Evaluation



Think immediately:

STACK



Next: ↳ [Daily Temperatures \(LC 739\)](#) — one of the most important **Monotonic Stack** questions. 🚀

daily temperature

739. Daily Temperatures

This is one of the most important **Monotonic Stack** questions.

1. Question Explanation

You are given an array:

```
</> C++
```



For each day, find:

How many days you have to wait until a warmer temperature occurs.

If no warmer temperature exists:

```
</> C++
```



Example

Input:

```
[73,74,75,71,69,72,76,73]
```



Output:

```
[1,1,4,2,1,1,0,0]
```

Why?

```
73 -> warmer temp 74 after 1 day
74 -> warmer temp 75 after 1 day
75 -> warmer temp 76 after 4 days
71 -> warmer temp 72 after 2 days
69 -> warmer temp 72 after 1 day
72 -> warmer temp 76 after 1 day
76 -> no warmer temperature
73 -> no warmer temperature
```



2. Brute Force Approach

Idea

For every day:

Look at all future days.

Find first temperature greater than current.

Example

73 74 75 71



For 73:

74 > 73



Answer = 1

For 75:

71 ✖
No more elements



Answer = 0

Time Complexity

Outer loop:

N



Inner loop:

N



Total:

$O(N^2)$



Space Complexity

$O(1)$



(ignoring answer array)

C++ Code

```
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {

        int n = temperatures.size();

        vector<int> ans(n,0);

        for(int i=0;i<n;i++) {

            for(int j=i+1;j<n;j++) {

                if(temperatures[j] > temperatures[i]) {

                    ans[i] = j-i;
                    break;
                }
            }
        }

        return ans;
    }
};
```

3. Better Approach

Observation

For every element we need:

```
Next Greater Element
```



to the right.

This is a famous stack pattern.

Idea

Store indices in stack.

While current temperature is greater than stack top temperature:

```
</> C++
```



we found the answer for stack top index.

Dry Run

```
[73,74,75]
```



i = 0

Push index 0

```
Stack: [0]
```



i = 1

```
74 > 73
```



Answer for day 0:

```
1 - 0 = 1
```



Pop.

Push 1.

i = 2

```
75 > 74
```



Answer for day 1:

```
2 - 1 = 1
```



Pop.

Push 2.

Remaining indices have answer 0.

Why Indices?

Because answer requires:

```
</> C++
```



Number of days.

Time Complexity

```
O(N)
```



Each index pushed once and popped once.

Space Complexity

$O(N)$



Stack.

C++ Code

```
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {

        int n = temperatures.size();

        vector<int> ans(n,0);

        stack<int> st;

        for(int i=0;i<n;i++) {

            while(!st.empty() &&
                temperatures[i] > temperatures[st.top()]) {

                int idx = st.top();
                st.pop();

                ans[idx] = i - idx;
            }

            st.push(i);
        }

        return ans;
    }
};
```

4. Optimal Approach

The monotonic stack solution is already optimal.

Reason:

Every element must be inspected at least once.

So:

$O(N)$



is the best possible.

Monotonic Decreasing Stack

Stack stores indices whose temperatures are in:

Decreasing Order



Example:

76
75
71
69



When a warmer temperature arrives:

72



it starts resolving previous days.

Visualization

Temp : 73 74 75 71 69 72 76 73
Index: 0 1 2 3 4 5 6 7



Stack stores unresolved indices.

Push 73
74 resolves 73



Push 74
75 resolves 74

Push 75
Push 71
Push 69

72 resolves 69
72 resolves 71

76 resolves 72
76 resolves 75

Time Complexity

$O(N)$



Space Complexity

$O(N)$



C++ Code

```
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temp) {

        int n = temp.size();

        vector<int> ans(n,0);

        stack<int> st;

        for(int i=0;i<n;i++) {

            while(!st.empty() &&
                temp[i] > temp[st.top()]) {
```

```
        ans[st.top()] = i - st.top();  
        st.pop();  
    }  
  
    st.push(i);  
}  
  
return ans;  
}  
};
```

Interview Notes

Pattern Recognition

Whenever you hear:

Next Greater Element
Next Smaller Element
Nearest Greater Right
Nearest Greater Left
Daily Temperatures
Stock Span
Histogram



Think immediately:

MONOTONIC STACK



Most Important Observation

Store:

INDEX



not temperature.

Because answer needs:

</> C++



Common Mistake

Wrong:

```
stack<int> st;
```

</> C++



Then you cannot compute:

</> C++



Correct:

</> C++



Next question: **Car Fleet (LC 853)**. This one introduces a very clever sorting + stack/greedy idea. 🚀

car fleet

853. Car Fleet

1. Question Explanation

There are `n` cars moving towards a target.

You are given:

```
target  
position[i]
```

`</>` C++



where:

- `position[i]` = current position of car
- `speed[i]` = speed of car
- all cars move towards the target

Fleet Definition

If a faster car catches a slower car before reaching the target:

They become one fleet



After joining:

They move together at the slower car's speed



Example

```
target = 12
```



```
position = [10,8,0,5,3]
```

```
speed    = [2,4,1,1,3]
```

Output:

3



Visualization

Target -> 12



Pos 10 Speed 2

Pos 8 Speed 4

Both reach target in:

$$(12-10)/2 = 1$$

$$(12-8)/4 = 1$$

Same arrival time.

Therefore:

1 Fleet



Key Observation

We don't care about where cars meet.

We only care about:

Time required to reach target



2. Brute Force Approach

Idea

For every car:

- Simulate movement.
- Check if it catches another car.
- Merge fleets.

This involves continuous movement simulation.

Time Complexity

$O(N^2)$



or worse depending on simulation.

Space Complexity

$O(1)$



C++ Code

Not usually implemented in interviews because simulation becomes messy and inefficient.

3. Better Approach

Observation

Compute time for each car:

`</> C++`



Example

target = 12



Position	Speed	Time
10	2	1
8	4	1
5	1	7
3	3	3
0	1	12

Sort by Position

Cars nearest to target should be processed first.

Position	Time
10	1
8	1
5	7
3	3
0	12



Fleet Rule

Moving from right to left:

If:

```
currentTime <= fleetTimeAhead
```



then current car catches fleet ahead.

So:

```
No new fleet
```



Else:

```
New fleet formed
```



Time Complexity

Sorting:

```
O(N log N)
```



Traversal:

$O(N)$



Total:

$O(N \log N)$



⌵ ⌶ ⌷ ⌸
☰ ChatGPT ⌵ ⬆ Upgrade

$O(N)$



C++ Code

```
class Solution {
public:
    int carFleet(int target, vector<int>& position,
                vector<int>& speed) {

        int n = position.size();

        vector<pair<int,double>> cars;

        for(int i=0;i<n;i++) {

            double time =
                (double)(target-position[i])/speed[i];

            cars.push_back({position[i],time});
        }

        sort(cars.begin(), cars.end());

        stack<double> st;

        for(int i=n-1;i>=0;i--) {

            double currTime = cars[i].second;

            if(st.empty() || currTime > st.top())
                st.push(currTime);
        }
    }
};
```

```
    }  
  
    return st.size();  
}  
};
```

4. Optimal Approach

The stack isn't actually needed.

We only need:

Last fleet time



Core Insight

After sorting by position:

Process from nearest car to farthest.

Keep:

</> C++



Case 1

currTime <= maxTimeSeen



Current car catches fleet ahead.

No new fleet



Case 2

```
currTime > maxTimeSeen
```



Current car cannot catch fleet ahead.

```
New fleet formed
```



Update:

```
</> C++
```



Dry Run

Position	Time
10	1
8	1
5	7
3	3
0	12



Start from right:

```
10 -> fleet count = 1  
maxTime = 1
```



```
8 -> 1 <= 1  
joins fleet
```



```
5 -> 7 > 1  
new fleet  
count = 2  
maxTime = 7
```




```
3 -> 3 <= 7  
joins fleet
```



```
0 -> 12 > 7  
new fleet  
count = 3
```



Answer:

3



Time Complexity

Sorting:

$O(N \log N)$



Traversal:

$O(N)$



Total:

$O(N \log N)$



Space Complexity

$O(N)$



(for storing cars)

C++ Code

```

class Solution {
public:
    int carFleet(int target,
                  vector<int>& position,
                  vector<int>& speed) {

        int n = position.size();

        vector<pair<int,double>> cars;

        for(int i=0;i<n;i++) {

            double time =
                (double)(target-position[i]) / speed[i];

            cars.push_back({position[i], time});
        }

        sort(cars.begin(), cars.end());

        int fleets = 0;
        double maxTime = 0;

        for(int i=n-1;i>=0;i--) {

            if(cars[i].second > maxTime) {

                fleets++;
                maxTime = cars[i].second;
            }
        }

        return fleets;
    }
};

```

Interview Notes

Most Important Observation

Forget positions after sorting.

Think only in terms of:



Why Process From Right to Left?

Because the car nearest to the target determines whether cars behind can catch it.

Pattern Recognition

Questions involving:

Sorting by position
Processing from one side
Merging groups/fleets



often use:

Greedy + Sorting



rather than a traditional stack.

Common Mistake

Many people think:

Faster speed = separate fleet



Wrong.

The only thing that matters is:

Time to reach target



If a car behind reaches earlier, it catches the fleet ahead and becomes part of it.

Next: ↪ [Largest Rectangle in Histogram \(LC 84\)](#) — arguably the most important monotonic stack problem and the parent of many hard stack questions. 🚀

84. Largest Rectangle in Histogram

This is one of the most important **Monotonic Stack** problems.

Many hard questions are derived from it:

- Maximal Rectangle
- Sum of Subarray Minimums
- Trapping Rain Water
- Stock Span
- Daily Temperatures

1. Question Explanation

You are given:

```
</> C++
```



where each value represents the height of a histogram bar.

Width of every bar is:

```
1
```



Find the **largest rectangular area** possible.

Example

Input:

```
[2,1,5,6,2,3]
```



Output:

10

Visualization

```
      |
      | 6
    | |
    |5|
    | |   |
  |2 | |2 |3
  -----
```



Using bars:

5 and 6



Minimum height:

5



Width:

2



Area:

$5 \times 2 = 10$



Answer = 10

2. Brute Force Approach

Idea

For every bar:

Assume it is the smallest bar.

Expand left and right until a smaller bar appears.

Compute area.

Example

[2,1,5,6,2]



Take height = 5

Expand:

5 6



Width:

2



Area:

$5 \times 2 = 10$



Time Complexity

For every index:

N



Expand:

N



Total:

$O(N^2)$



Space Complexity

$O(1)$



C++ Code

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {

        int n = heights.size();
        int ans = 0;

        for(int i=0;i<n;i++) {

            int left = i;
            int right = i;

            while(left-1 >= 0 &&
                  heights[left-1] >= heights[i])
                left--;

            while(right+1 < n &&
                  heights[right+1] >= heights[i])
                right++;

            int width = right - left + 1;

            ans = max(ans, heights[i] * width);
        }

        return ans;
    }
};
```

3. Better Approach

Observation

For each bar we need:

NSL

Nearest Smaller to Left

NSR

Nearest Smaller to Right

Because:

Rectangle can extend
until smaller element appears



Formula

For every index:

```
</> C++
```



```
</> C++
```



Example

[2,1,5,6,2,3]



For height = 5

NSL = index 1
NSR = index 4



Width:

4 - 1 - 1 = 2



Area:

$$5 \times 2 = 10$$



Time Complexity

Finding NSL:

$O(N)$



Finding NSR:

$O(N)$



Area calculation:

$O(N)$



Total:

$O(N)$



Space Complexity

$O(N)$



(NSL + NSR + Stack)

C++ Code

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {

        int n = heights.size();
```

```
vector<int> left(n), right(n);

stack<int> st;

// NSL

for(int i=0;i<n;i++) {

    while(!st.empty() &&
           heights[st.top()] >= heights[i])
        st.pop();

    left[i] = st.empty() ? -1 : st.top();

    st.push(i);
}

while(!st.empty()) st.pop();

// NSR

for(int i=n-1;i>=0;i--) {

    while(!st.empty() &&
           heights[st.top()] >= heights[i])
        st.pop();

    right[i] = st.empty() ? n : st.top();

    st.push(i);
}

int ans = 0;

for(int i=0;i<n;i++) {

    int width = right[i] - left[i] - 1;

    ans = max(ans,
               heights[i] * width);
}

return ans;
}
};
```

4. Optimal Approach

Key Observation

We don't need separate NSL and NSR arrays.

We can calculate area immediately while popping.

Monotonic Increasing Stack

Store indices in increasing height order.

Stack:



1
2
5
7

Increasing.

Why Pop?

Whenever:

```
</> C++
```



we found the first smaller element on the right.

Now we can calculate area for the popped bar.

Magic Formula

Suppose:

Current Index = i



Popped Height = h

After popping:

```
Stack Top = left smaller
```



Current index:

```
right smaller
```



Width:

```
</> C++
```



Area:

```
</> C++
```



Dry Run

```
[2,1,5,6,2,3]
```



Stack

```
2
```



See 1:

```
1 < 2
```



Pop 2.

Width = 1

Area = 2

Push 1.

Push 5.

Push 6.

See 2.

2 < 6



Pop 6.

Width = 1

Area = 6

Still:

2 < 5



Pop 5.

Width = 2

Area = 10

Maximum becomes 10.

Trick

Append a fake:

```cpp

0



at the end.

This forces all remaining bars to pop.

---

## Time Complexity

Each index:

- pushed once
- popped once

Therefore:

$O(N)$



## Space Complexity

$O(N)$



## C++ Code (Most Asked In Interviews)

```
class Solution {
public:
 int largestRectangleArea(vector<int>& heights) {

 int n = heights.size();

 stack<int> st;
 int ans = 0;

 for(int i=0;i<=n;i++) {

 int currHeight = (i == n) ? 0 : heights[i];

 while(!st.empty() &&
 heights[st.top()] > currHeight) {

 int h = heights[st.top()];
 st.pop();

 int width;

 if(st.empty())
 width = i;
 }
 }
 }
};
```

```
 else
 width = i - st.top() - 1;

 ans = max(ans, h * width);
 }

 st.push(i);
}

return ans;
}

};
```

---

## Interview Notes

### Most Important Formula

When popping:

</> C++



</> C++



</> C++



---

### Pattern Recognition

If question asks:

Nearest Smaller Left  
Nearest Smaller Right  
Maximum Rectangle  
Range where element is minimum



Think:

MONOTONIC STACK



## Common Mistake

Many people use:

`</>` C++



```
while(heights[st.top()] >= currHeight)
```

instead of

`</>` C++



```
while(heights[st.top()] > currHeight)
```

For this particular one, both can work with proper handling, but mixing conventions often causes width calculation bugs.

## Must Remember

For interviews, memorize this pattern:

`</>` C++



```
while(!st.empty() &&
 heights[st.top()] > currHeight)
{
 // calculate area
}
```

This exact template appears repeatedly in hard stack problems. 🚀